

NAME – Vedant Vijay Khedekar

PRN – 202501100090

DIV – SE1

BATCH – S15

DATASET NAME – COVID-19 DATASET

The COVID-19 Dataset contains information such as date, confirmed cases, deaths, recovered cases, active cases, testing count, and vaccination data. This dataset is used to analyze pandemic trends and healthcare response effectiveness. NumPy is used for numerical operations like calculating mean, sum, growth rates, and sorting values efficiently. Pandas is used for data manipulation, such as filtering by date range, grouping by region, and modifying the dataset. By using both libraries, we can easily analyze and extract useful insights from the data. This helps in better understanding of disease spread patterns and vaccine administration in healthcare applications.

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
```

Step 2: Create Sample COVID-19 Dataset

```
data = { 'Date': ['2020-03-01', '2020-03-05', '2020-03-10', '2020-03-15', '2020-03-20'], 'Confirmed_Cases': [100, 500, 2000, 5000, 12000], 'Deaths': [5, 30, 150, 400, 1000], 'Recovered': [20, 150, 800, 2000, 5000], 'Active_Cases': [75, 320, 1050, 2600, 6000], 'Tests_Conducted': [500, 3000, 15000, 40000, 80000] }
df = pd.DataFrame(data)
print(df)
```

➤ NUMPY OPERATIONS (10 Methods)

```
# Convert columns to NumPy arrays
confirmed =
np.array(df['Confirmed_Cases']) deaths
= np.array(df['Deaths']) tests =
np.array(df['Tests_Conducted'])
```

```
Given: Confirmed Cases = [100, 500, 2000, 5000, 12000]
Deaths = [5, 30, 150, 400, 1000]
Tests = [500, 3000, 15000, 40000, 80000]
```

1. Mean Confirmed Cases

```
print(np.mean(confirmed))
```

OUTPUT: 4000.0

2. Median Deaths

```
print(np.median(deaths))
```

OUTPUT: 150.0

3. Standard Deviation of Cases

```
print(np.std(confirmed))
```

OUTPUT: $\approx$ 4830.46

4. Maximum Cases

```
print(np.max(confirmed))
```

OUTPUT: 12000

5. Minimum Deaths

```
print(np.min(deaths))
```

OUTPUT: 5

6. Total Tests Conducted

```
print(np.sum(tests))
```

OUTPUT: 138500

7. Square of Confirmed Cases

```
print(np.square(confirmed))  
OUTPUT: [10000, 250000,  
4000000,      25000000,  
144000000]
```

8. Log of Tests

```
print(np.log(tests))
```

**OUTPUT: [6.215, 8.007,
9.616, 10.597, 11.290]
(approx)**

9. Sort Deaths in Ascending Order

```
print(np.sort(deaths))
OUTPUT: [5, 30, 150, 400,
1000]
```

10. Index of Maximum Confirmed Cases

```
print(np.argmax(confirmed
))
```

OUTPUT: 4

➤ PANDAS OPERATIONS (10 Methods)

Date	Confirmed	Deaths	Active	Recovered	Tests
2020-03-01	100	5	75	20	500
2020-03-05	500	30	320	150	3000
2020-03-10	2000	150	1050	800	15000
2020-03-15	5000	400	2600	2000	40000
2020-03-20	12000	1000	6000	5000	80000

1. Display first rows

```
print(df.head())
OUTPUT: (Displays all 5 rows of dataset)
```

2. Dataset Info

```
print(df.info())
OUTPUT: RangeIndex: 5 entries, 0 to 4 Columns: 6
entries Data types: int64, object
```

3. Statistical Summary

```
print(df.describe())
OUTPUT: count: 5.0, mean: 4000.0, std: 4830.46, min:
100.0, max: 12000.0
```

4. Sort by Confirmed Cases (descending)

```
print(df.sort_values(by='Confirmed_Cases',
ascending=False))
OUTPUT: 12000, 5000, 2000, 500, 100
```

5. Filter cases > 1000

```
print(df[df['Confirmed_Cases'] > 1000]) OUTPUT: 2020-
03-10: 2000 cases 2020-03-15: 5000 cases 2020-03-20:
12000 cases
```

6. Add Case Fatality Rate Column

```
df['Fatality_Rate'] =
(df['Deaths']/df['Confirmed_Cases'])*100
OUTPUT: 2020-03-01: 5.0% 2020-03-05: 6.0% 2020-03-10:
7.5%
```

7. Group by Week

```
print(df.groupby(df.index//2)['Confirmed_Cases'].sum())
```

OUTPUT: Week 1: 600 cases Week 2: 7000 cases Week 3: 12000 cases

8. Check Null Values

```
print(df.isnull().sum())
```

OUTPUT: All columns: 0 (no missing values)

9. Calculate Recovery Rate

```
df['Recovery_Rate'] =  
(df['Recovered']/df['Confirmed_Cases'])*100
```

OUTPUT: 2020-03-01: 20.0% 2020-03-05: 30.0% 2020-03-10: 40.0%

10. Drop Deaths Column

```
df = df.drop(columns=['Deaths'])
```

OUTPUT: Remaining columns: Date, Confirmed_Cases, Recovered, Active_Cases, Tests_Conducted